Unit 8 Bash Shell Basics

Course code LX13
Linux Basics

Objectives

After completing this unit, students should be able to:

- Use wildcards to access files with similar names.
- Use redirection and pipes to control the input and output of processes.
- Use line continuation to enter commands that span the command line.
- Group commands in order to control their execution

The Shell

- User interface to Linux
- Command interpreter
- Enables multiple tasks
- Comprehensive programming language
- Shells available: csh, tcsh, pdksh, ash, sash, zsh, bash

- Default in Linux: **bash** (Bourne Again SHell)

Metacharacters and Wildcards

- Metacharacters are characters that the shell interprets as having a special meaning.

  Examples:

  `< > | ; ! ? * [ ] $ \ " ' ` ~ ( ){}`

- Wildcards are a subset of metacharacters that are used to search for and match file patterns.

  Examples:

  `? * [] [-]`

Character Substitution in File Names

- Wildcards: **?** *****
- One character compare

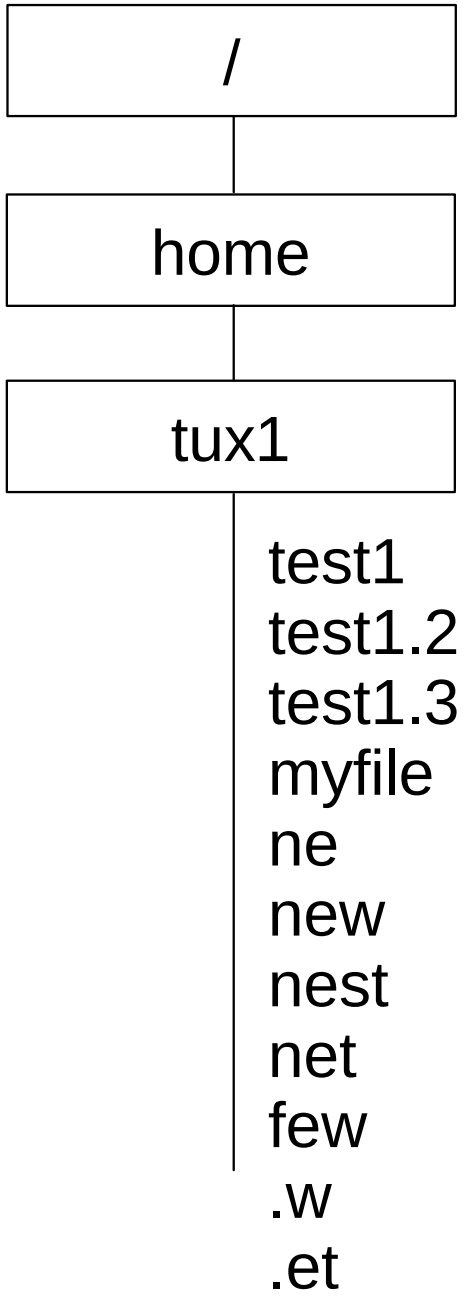
```
$ ls ne?
net  new

$ rm ?e?
few  net  new
```

- Multiple character compare

```
$ cp n* /tmp
ne  net  new  nest

$ lpr *w
new  few
```



Character Substitution in File Names *(2)*

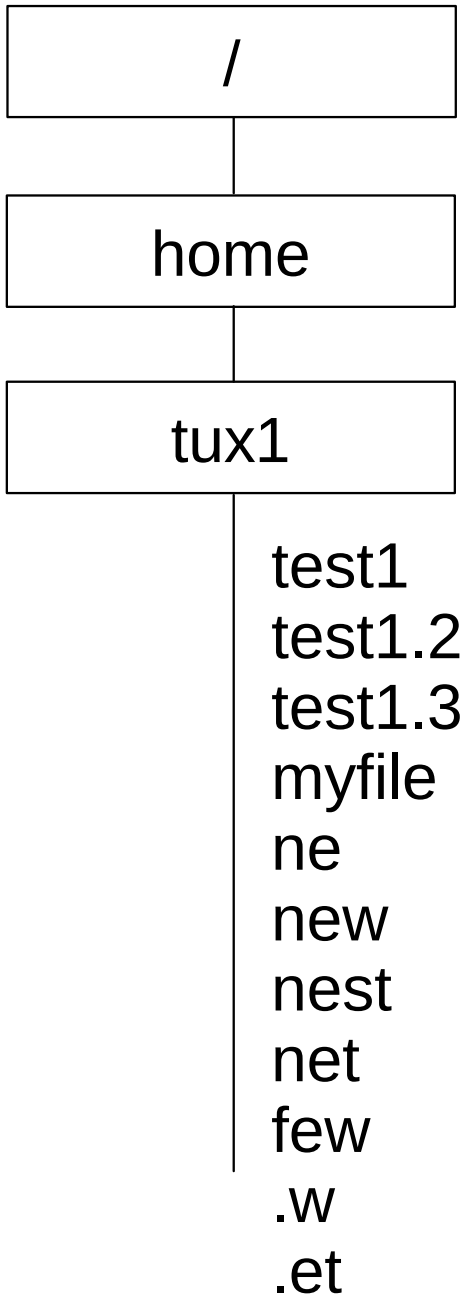
- Inclusive list: `[ ]` `[ - ]` `[! ]`

```
$ ls ne[stw]
net  new

ls *[1-5]
test1  test1.2
test1.3

$ lpr [!tn]*
myfile few

cat ?[!y]*[2-5]
test1.2  test1.3
```



The { } metacharacters

- The { } expansion creates arbitrary strings
- Can be used in pathnames

- Basic example

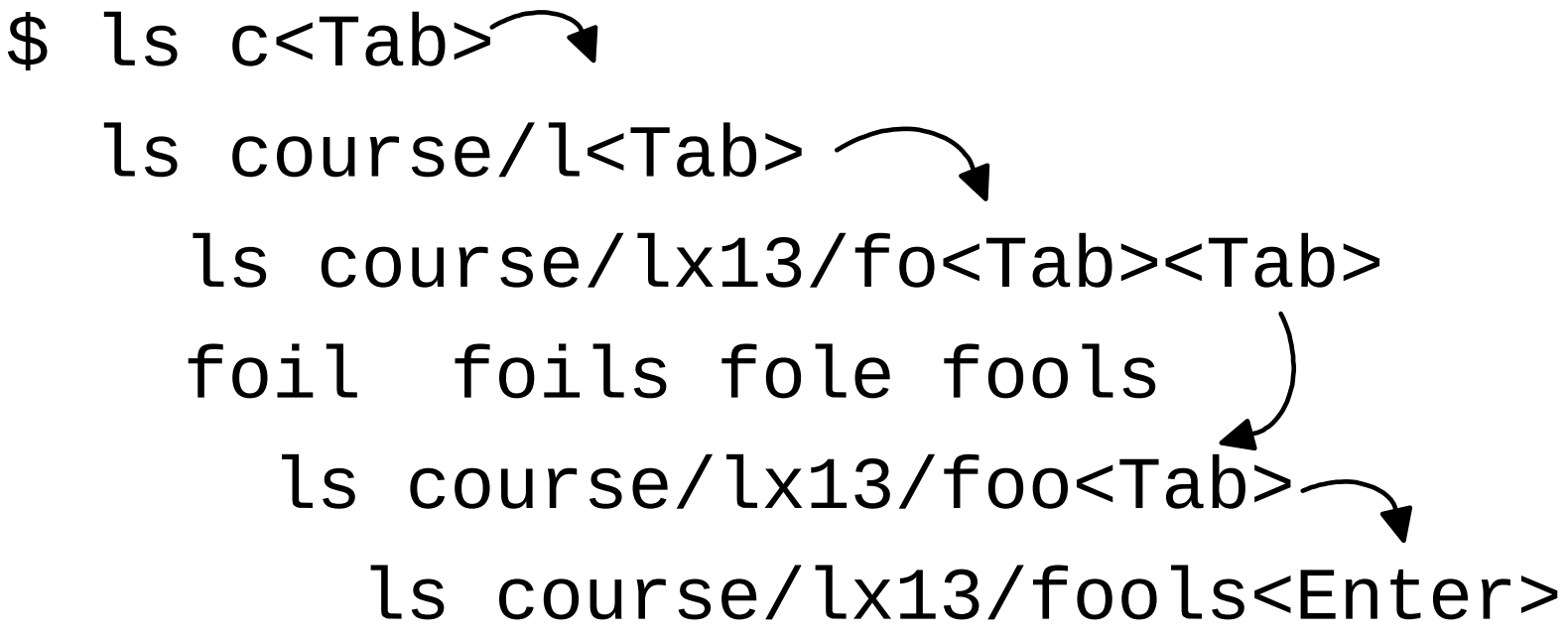
```
$ pwd
/home/tux6
$ ls course/lx1[34]/{foils,presentation}
```

- Advanced Example

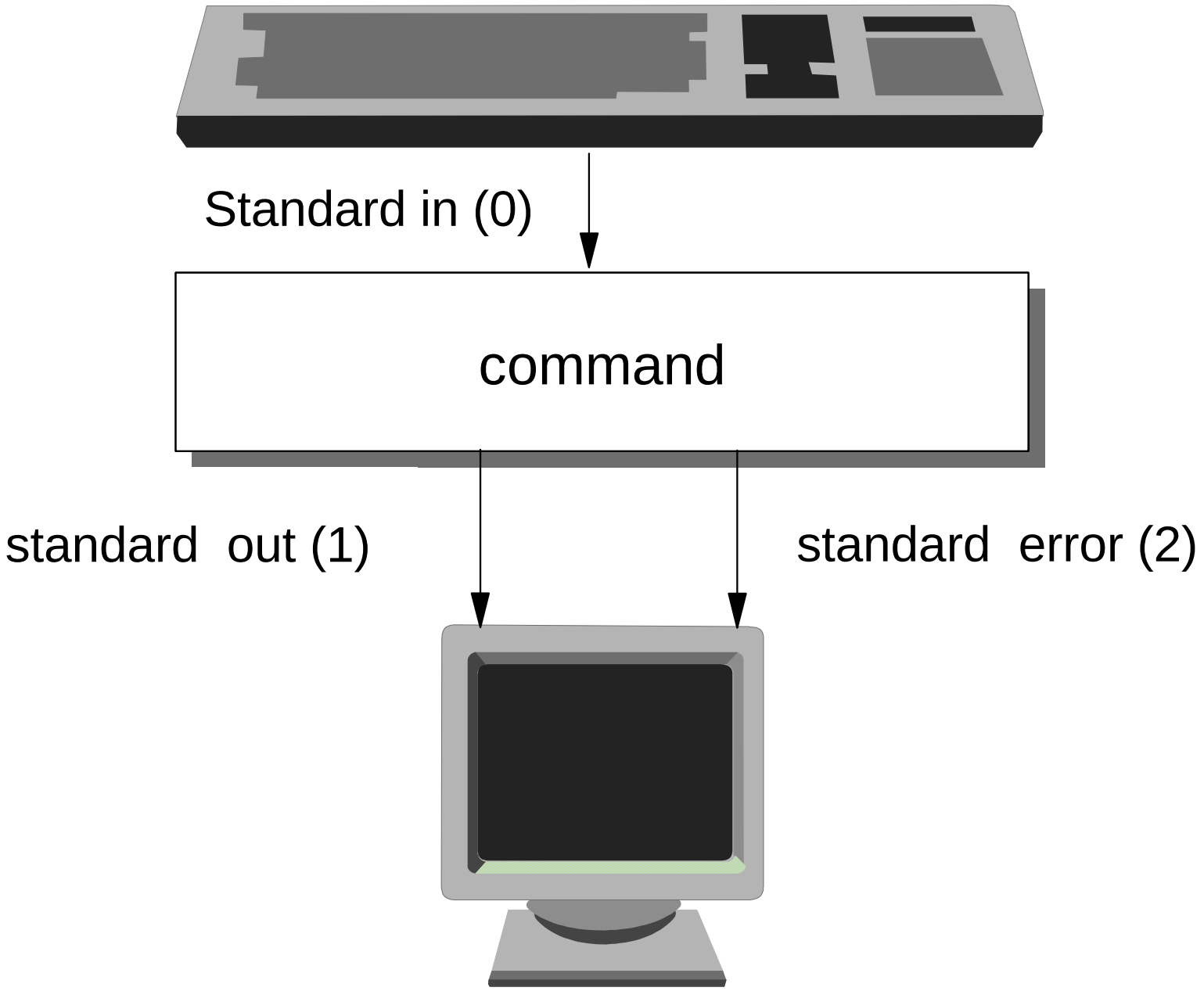
```
$ ls
course/lx13/{old/{f*,p*},new/{f*l,p*.?*}}
```

File name completion

- What do you do when you get tired of typing complete pathnames?
- You use **File name completion**



The Standard File Descriptors



File Descriptors

- Every program has a file descriptor associated with it
- Three descriptors are assigned by the shell when the program starts (STDIN, STDOUT and STDERR)
- Other descriptors are assigned by the program when it opens files

Standard In	STDIN	<	0
Standard Out	STDOUT	>	1
Standard Error	STDERR	2>	2

Input Redirection

- Default Standard Input:

```
$ cat
Amsterdam
Amsterdam
Utrecht
Utrecht
<ctrl-d>
```

- Redirect from a file:

```
$ cat < cities
Amsterdam
Utrecht
```

Output Redirection

- Default Standard Output:
  $ ls
  file1 file2 file3

- Redirect output to a file:
  $ ls **>** ls.out

- Redirect and append output to a file:
  $ ls **>>** ls.out

- Create a file with redirection:
  $ cat > new_file
  save this line
  <ctrl-d>

Error Redirection

- Default Standard Error:
```
$ cat filea
cat: filea: No such file or directory
```
- Redirecting error output to a file:
```
$ cat filea 2> error.file
$ cat error.file
cat: filea: No such file or directory
```
- Redirect and append error output to a file:
```
$ cat filea 2>> error.file
```
- Discard error output:
```
$ cat filea 2> /dev/null
```

Combined Redirection

- Combined redirects

```
$ command < infile > outfile 2> error.file
$ command >> appendfile 2>> error.file < infile
```

- Association

```
$ command > outfile 2>&1
```

This redirects stderr to the same file as stdout (outfile).

This is NOT the same as

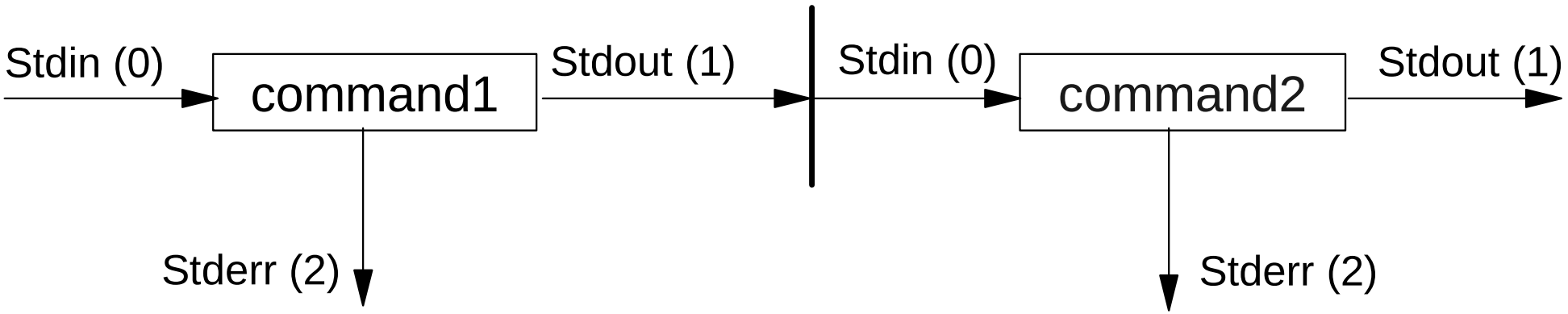
```
$ command 2>&1 > outfile
```

This first redirects stderr to stdout (screen) and then stdout to outfile. Not stderr to outfile!!!

Pipes

- A sequence of two or more commands separated by a vertical bar (|) is called a **pipe**. The standard output of each command becomes the standard input of the next command.

```
$ ls -l | wc -l
```



Filters

- A filter is a command that reads from standard in, transforms the input in some way and writes to standard out. They can, therefore, be used at intermediate points in a pipeline.

- `$ ls | grep ^n | wc -l`

Common Filters

- **expand**: Change tabs to spaces
- **sed**: Allows string substitutions
- **awk**: Pattern scanning and processing
- **fmt**: Insert line wraps so text looks pretty
- **tac**: Display lines in reverse order
- **tr**: Substitute characters
- **grep**: Only displays lines that match a pattern
- **nl**: Number lines
- **pr**: Format for printer

Split Output

- The **tee** command reads standard input and sends the data to both standard out and a file.

```
$ ls | wc -l
  3

$ ls | tee ls.save | wc -l
  3

$ cat ls.save
file1
file2
file3
```

Command Grouping

- Multiple commands can be entered on the same line, separated by a semi-colon (;)

```
$ date ; pwd
```

is equivalent to entering:

```
$ date
$ pwd
```

There is no relationship between the output of cmd1 and the input to cmd2.

Line Continuation

- The backslash (\) can be used to continue a command on a separate line. A secondary prompt (">") is issued by the shell to indicate line continuation.

```
$ cat /home/john/mydir/mysudir/data/information/letter\
> /pictures/logo.jpg
```

Checkpoint

1. What will the following command match?

```
$ ls ???[!a-z]*[0-9]t
```

2. Place the output of the `cat filea` command in fileb and the errors in filec

Checkpoint *(2)*

3. What will the following command do?

```
$ cat namesfile | wc -l > number
```

Unit Summary

- Metacharacters are a number of characters that have a special meaning to the shell.

- Metacharacters are:
  ! " $ % ^ & * ( ) { } [ ] # ~ ; ' < > / ? ` \ |

- Wildcards are a subset of the metacharacters that work on file names
  ? * [ ]